

RoboPet - Status Report & Dog Interactions

Project status report + a ready-to-build plan for dog interactions (tap the dog -> Feed / Pet / Charge).
Written to be handed directly to Cursor.

Cursor task (build this now)

Implement the following in order. Do not delete any decor code.

1. Freeze the decor UI. Add a global const bool kDecorEnabled = false; and gate the Decorate button with it (if (kDecorEnabled) DecorateButton()). Keep all existing decor/shop code untouched.
2. Make the dog tappable. Add TapCallbacks to RobotDog. On onTapUp, if not in decorate mode, call game.openDogActions().
3. Add the action menu. Create a DogActionBar overlay (Feed / Pet / Charge) pinned near the bottom center.
4. Wire the actions. Feed -> care.feed() + dog.reactEat(); Pet -> care.pet() + dog.reactHappy(); Charge -> care.recharge() + dog.reactHappy(). Close the menu after each tap.
5. Add care methods. Add feed(), pet(), recharge() to the existing CareNotifier (adapt field names to the real CareState), each persisting to Isar.
6. Sync animation to care. Add RobotDog.applyCareState(...) and a ref.listen(careProvider, ...) so the dog shows sleep / lowPower based on state.
7. Register the overlay. Add 'dogActions': (ctx, g) => DogActionBar(game: g) to the game's overlayBuilderMap.

Acceptance test

- Tap the dog -> action menu appears (and not while in decorate mode)
- Feed -> dog plays the eat animation and hunger goes up
- Pet / Charge -> happy animation, matching stat goes up
- Menu closes after an action; dog returns to idle and resumes wandering
- Restart the app -> care values persist (Isar)
- Decorate button is hidden (kDecorEnabled = false) but no code was removed

1) Overview

- Hybrid Tamagotchi + RPG mobile game (Flutter + Flame)
- Riverpod, Isar (offline-first care/decay), Firestore (economy), async-ghost PvP
- iOS 15+ / Android 8.0+

2) Completed

- Phase 1 - Care/decay (Isar) + base UI
- Phase 1.5 - gamification layer
- Phase 2 - Economy (scrap/rareCogs/premiumGems) + module slots
- Phase 3 - Combat (working: tick, damage, ghost PvP)
- Art pass - PixelLab dog sprite + brick wall (started)
- Phase 4 - Decor + Shop placeholder loop (integrated by Cursor)

3) Current state

- Working: room navigation, dog wander, decor buy -> place -> drag -> persist (placeholder boxes), bottom shop strip.
- Visuals: dog sprite + brick wall exist; floor is code (SimpleFloor); decor images not made yet (placeholder boxes).
- Note: you are not happy with the Decorate / shop UI - addressed in section 7 (freeze it, do not delete).

4) Open / pending

- Dog interactions (tap -> feed/pet/charge) - none yet, building now
- Real decor sprites (assets/images/decor/<id>.png)
- Wall PNG regenerated ideally at 200x175 (to avoid brick stretch)
- Dog frame PNGs (idle/walk/eat/happy/sleep/low_power/attack folders) + _config counts
- Decor UI decision (keep / simplify / remove)
- (Later) Sudoku minigame, ESP32 hardware

5) Dog interactions - code

Tap the dog -> small action menu (Feed / Pet / Charge) -> care state updates + dog plays the matching animation (eat / happy) -> returns to idle. Care already exists from Phase 1; we only add the trigger + animation reaction.

5.1 RobotDog - tap + reaction

```
class RobotDog extends SpriteAnimationGroupComponent<DogAnim>
  with HasGameReference<RoboPetGame>, TapCallbacks {
  RobotDog({required Vector2 position})
    : super(position: position, size: Vector2.all(160),
      anchor: Anchor.bottomCenter, priority: 10);

  bool _asleep = false;

  // Play an animation for a fixed duration, then return to idle
  Future<void> reactFor(DogAnim anim, double seconds) async {
    if (_asleep) return;
    stopWander();
    current = anim;
    await Future<void>.delayed(
      Duration(milliseconds: (seconds * 1000).round()));
```

```

        current = DogAnim.idle;
        startWander();
    }

    Future<void> reactEat()    => reactFor(DogAnim.eat, 2.5);
    Future<void> reactHappy() => reactFor(DogAnim.happy, 2.0);

    @override
    void onTapUp(TapUpEvent event) {
        if (game.decorateMode) return;    // ignore taps while decorating
        game.openDogActions();
    }
}

```

5.2 Action menu overlay

```

class DogActionBar extends ConsumerWidget {
    const DogActionBar({super.key, required this.game});
    final RoboPetGame game;

    @override
    Widget build(BuildContext context, WidgetRef ref) {
        final care = ref.read(careProvider.notifier);
        Widget btn(String icon, String label, VoidCallback onTap) =>
            GestureDetector(
                onTap: () { onTap(); game.closeDogActions(); },
                child: Column(mainAxisSize: MainAxisSize.min, children: [
                    Text(icon, style: const TextStyle(fontSize: 28)),
                    Text(label, style: const TextStyle(
                        color: Colors.white, fontSize: 12)),
                ]),
            );
        return Align(
            alignment: Alignment.bottomCenter,
            child: Container(
                margin: const EdgeInsets.only(bottom: 90),
                padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 14),
                decoration: BoxDecoration(
                    color: const Color(0xEE0B0E1A),
                    borderRadius: BorderRadius.circular(20),
                    border: Border.all(color: const Color(0xFF22E1FF)),
                ),
                child: Row(mainAxisSize: MainAxisSize.min, children: [
                    btn('food', 'Feed', () { care.feed(); game.dog.reactEat(); }),
                    const SizedBox(width: 28),
                    btn('pet', 'Pet', () { care.pet(); game.dog.reactHappy(); }),
                    const SizedBox(width: 28),
                    btn('bolt', 'Charge', () { care.recharge(); game.dog.reactHappy(); }),
                ]),
            ),
        );
    }
}

// inside RoboPetGame
void openDogActions() => overlays.add('dogActions');
void closeDogActions() => overlays.remove('dogActions');
// register: overlayBuilderMap -> 'dogActions': (ctx, g) => DogActionBar(game: g)

```

5.3 Care integration (adapt to your CareNotifier)

```

// CareNotifier (Phase 1) - adapt field names to your real CareState
void feed()    => _bump(hunger: 30);

```

```

void pet()      => _bump(happiness: 20);
void recharge() => _bump(energy: 40);

void _bump({int hunger = 0, int happiness = 0, int energy = 0}) {
  state = state.copyWith(
    hunger:      (state.hunger + hunger).clamp(0, 100),
    happiness:   (state.happiness + happiness).clamp(0, 100),
    energy:      (state.energy + energy).clamp(0, 100),
    lastInteraction: DateTime.now(),
  );
  _persist(); // write to Isar
}

```

5.4 Animation sync (battery / sleep)

```

// call when care changes:
// ref.listen(careProvider, (_, s) => dog.applyCareState(
//   energy: s.energy, asleep: s.asleep));
void applyCareState({required int energy, required bool asleep}) {
  _asleep = asleep;
  if (asleep) { current = DogAnim.sleep; stopWander(); return; }
  if (energy < 20) {
    current = DogAnim.lowPower;
  } else if (current == DogAnim.sleep || current == DogAnim.lowPower) {
    current = DogAnim.idle; startWander();
  }
}

```

Optional polish: a floating TextComponent (e.g. +30) above the dog when fed, plus a small heart effect. ~10 lines, add later.

6) What we should do - recommended order

8. NOW: Dog interactions (the code in this report). The heart of the game; gives the strongest alive feel.
9. Decor UI decision: section 7 - freeze with a feature flag for now (do not delete).
10. Load dog frame PNGs -> see real idle/walk/eat/happy animations.
11. Then: real decor sprites or a new feature (Sudoku minigame).

Recommendation: Dog interactions + real dog animations have the highest impact-to-effort ratio. Freeze decor for now to remove the clutter.

7) Decor UI - 3 options

- Keep: small polish (price tag, owned badge, remove button)
- Simplify: hide the Decorate button, move the shop into the main menu
- Remove (recommended): kDecorEnabled = false feature flag - the Decorate button is hidden, all code stays, re-enable anytime

```

const bool kDecorEnabled = false; // set true to bring Decorate back
// in NavBar/HUD:
if (kDecorEnabled) DecorateButton(),

```